

Recordatorio a la gestión de calidad

La gestión de calidad se refiere a todos los procesos, normas y actividades que aseguran que un proyecto de software cumpla con los estándares establecidos. No se trata solamente de que el sistema funcione, sino de que funcione correctamente, de forma eficiente y que realmente satisfaga las necesidades del usuario.

Imaginar panoramas donde una aplicación bancaria se equivoca en los montos o pierde información, las consecuencias serían graves. Generaría desconfianza, pérdidas económicas y posiblemente problemas legales. Por eso, la calidad no es opcional en el desarrollo de software, es un elemento **esencial**.

Plan de calidad del software

Para poder asegurar la calidad, no basta con tener buenas intenciones. Es necesario establecer un plan.

El plan de calidad del software es un documento en el que se define cómo se va a garantizar la calidad durante el desarrollo del proyecto. En este plan se establecen los objetivos de calidad, las responsabilidades de cada integrante del equipo y los procesos que se van a seguir para revisar y validar el producto.

Por ejemplo, un equipo puede establecer que ninguna funcionalidad será entregada sin haber pasado previamente por pruebas. Esa es una regla clara de calidad.

Si un equipo de trabajo no tiene reglas ni procesos definidos para asegurar la calidad probablemente generará errores constantes, retrasos en la entrega y problemas con el cliente.

El plan de calidad ayuda precisamente a evitar ese tipo de situaciones.

Estándares y métricas de calidad

Existen modelos y estándares que ayudan a las organizaciones a mejorar sus procesos de desarrollo de software.

Dos de los más importantes son CMMI y MoProSoft.

CMMI es un modelo que evalúa el nivel de madurez de los procesos de una organización. Tiene varios niveles, desde procesos básicos hasta procesos optimizados. Mientras más alto es el nivel, más control y calidad tiene la empresa en sus proyectos.

Por otro lado, MoProSoft es un modelo mexicano que está diseñado para mejorar los procesos de desarrollo en empresas de software, especialmente en pequeñas y medianas organizaciones.

Además de los estándares, también se utilizan métricas.

Las métricas nos permiten saber si realmente estamos mejorando o si hay problemas que debemos corregir.

Podemos medir la cantidad de errores, el tiempo de respuesta, la disponibilidad del sistema o incluso la satisfacción del usuario.

Impacto de la calidad en tiempo, costo y alcance

La calidad tiene un impacto directo en tres aspectos fundamentales de cualquier proyecto: el tiempo, el costo y el alcance.

Cuando hay problemas de calidad, normalmente se tiene que volver a hacer el trabajo. Esto significa que el equipo tiene que volver a hacer cosas que ya estaban terminadas. Eso provoca retrasos en el tiempo, incrementa los costos y muchas veces reduce el alcance del proyecto, porque ya no hay suficiente tiempo o recursos para cumplir con todo lo planeado.

Un ejemplo claro es cuando un software se lanza con **errores** y luego necesita múltiples **actualizaciones** para corregirlos. Eso implica más tiempo y más dinero.

¿acaso les ha pasado esto?

Control del cambio

En cualquier proyecto, los cambios son inevitables. El cliente puede solicitar nuevas funcionalidades, modificar requisitos o ajustar detalles.

El problema no es que haya cambios, sino cómo se gestionan esos cambios.

El control del cambio es el proceso mediante el cual se analizan, evalúan y aprueban las modificaciones dentro del proyecto. Esto implica revisar el impacto del cambio en el tiempo, el costo y el alcance antes de tomar una decisión.

Si el cliente cambia constantemente los requisitos y el equipo acepta todo sin control el proyecto se vuelve desordenado, se retrasa y puede incluso fracasar.

Es importante recordar una idea clave: la **calidad** no se revisa al final del proyecto, sino que se construye desde el inicio y se mantiene durante **todo el proceso**.

Objetivo del proyecto

El primer paso es definir el objetivo. El objetivo del proyecto debe ser claro y específico. No basta con decir “vamos a hacer un sistema”. No es lo mismo decir “crear una aplicación” que decir “desarrollar una aplicación que permita gestionar inventarios y reducir errores en un 30% en un periodo de seis meses”.

Para definir buenos objetivos se utiliza el modelo SMART, que indica que deben ser específicos, medibles, alcanzables, relevantes y con un tiempo definido.

Si un proyecto no tiene un objetivo claro probablemente el equipo no sabría hacia dónde dirigirse y el resultado sería confuso o incompleto.

Estimación de tiempo

La estimación del tiempo es uno de los aspectos más difíciles en la gestión de proyectos. Generalmente los proyectos no siempre se terminan en el tiempo planeado. Esto ocurre porque no se consideran factores como errores, cambios o problemas técnicos.

Existen diferentes formas de estimar el tiempo:

- Una de ellas es la estimación por analogía, que consiste en basarse en proyectos anteriores.
- Otra forma es dividir el proyecto en tareas más pequeñas y asignar un tiempo a cada una.

Por ejemplo, se puede dividir en diseño, desarrollo y pruebas, asignando una duración a cada fase. Sin embargo, siempre es importante considerar posibles imprevistos.

Estimación de costos

La estimación de costos permite calcular cuánto dinero se necesita para llevar a cabo el proyecto. Los costos pueden incluir salarios del equipo, herramientas de software, equipo de hardware y otros **gastos administrativos**.

Existen dos enfoques principales:

- El primero es de arriba hacia abajo, donde se define un presupuesto total y luego se distribuye.
- El segundo es de abajo hacia arriba, donde se calcula el costo de cada elemento y luego se suman.

Si un proyecto calcula mal sus costos podría quedarse sin recursos antes de terminar o volverse financieramente inviable.

Estimación de personal requerido

En un proyecto no todas las personas cumplen la misma función. Es importante definir cuántas personas se necesitan y qué roles tendrán.

Por ejemplo, puede haber desarrolladores, diseñadores, testers y un líder de proyecto.

Si hay pocas personas en un proyecto el trabajo se podría ver afectado y retrasar. En cambio, si hay demasiadas, puede haber problemas de coordinación.

Análisis de riesgos

Todos los proyectos tienen **riesgos**. Un riesgo es cualquier evento que puede afectar negativamente el proyecto. Un riesgo puede ser un problema técnico, una falta de recursos o cambios en los requisitos.

Por ejemplo, un riesgo técnico puede ser que una tecnología no funcione como se esperaba.

El análisis de riesgos implica identificarlos, evaluar su impacto y definir estrategias para enfrentarlos.

Piensa en un riesgo en una app de la actualidad, ¿Qué riesgos creen que tendría una aplicación como Uber?

Evaluación del riesgo

No todos los riesgos tienen la misma importancia. Algunos son más probables y otros tienen mayor impacto. Para ello se utilizan herramientas como la **matriz de riesgos**, en donde se clasifican según su probabilidad y su impacto.

Un riesgo con alta probabilidad y alto impacto debe ser atendido con prioridad.

Estrategias frente al riesgo

En la gestión de proyectos de software, los riesgos son situaciones o eventos que pueden afectar negativamente el desarrollo del proyecto. Estos riesgos pueden provocar retrasos, incremento de costos, fallas técnicas o problemas en el cumplimiento de los objetivos.

Para reducir el impacto de los riesgos, existen diferentes estrategias de manejo:

1. **Evitar el riesgo.** Consiste en eliminar la causa del problema o modificar el plan para que el riesgo no ocurra.

Ejemplo: Si una tecnología es muy inestable o desconocida para el equipo, se puede reemplazar por otra más confiable.

2. **Mitigar el riesgo.** Busca reducir la probabilidad de que el riesgo ocurra o disminuir su impacto.

Ejemplo: Realizar pruebas frecuentes para detectar errores antes de que afecten todo el sistema.

3. **Transferir el riesgo.** Consiste en pasar la responsabilidad del riesgo a un tercero.

Ejemplo: Contratar un servicio externo de hosting o soporte técnico especializado.

4. **Aceptar el riesgo.** Se utiliza cuando el impacto del riesgo es bajo o no es posible evitarlo. En este caso, el equipo decide asumir las consecuencias si el riesgo ocurre.

Ejemplo: Aceptar pequeños retrasos ocasionados por ajustes menores solicitados por el cliente.

Importancia de las estrategias frente al riesgo

Aplicar estrategias de manejo de riesgos permite:

- reducir problemas durante el proyecto
- mejorar la toma de decisiones
- anticipar posibles dificultades
- proteger el tiempo, costo y alcance del proyecto
- aumentar las probabilidades de éxito

En metodologías ágiles como **Scrum**, los riesgos suelen identificarse durante cada **sprint** mediante el seguimiento continuo del avance del proyecto y la detección de impedimentos o problemas técnicos.

Análisis de la viabilidad del proyecto

Antes de iniciar un proyecto, es importante analizar si realmente es viable.

Esto incluye evaluar si es posible técnicamente, si es rentable económicamente y si puede implementarse en la organización.

Por ejemplo, desarrollar una aplicación sin usuarios potenciales no sería viable.

Trabajo a entregar:

Teniendo en cuenta su proyecto integrador y la documentación previamente elaborada, entregar un trabajo en donde establezcan su objetivo, estimen tiempos, costos, personal, analicen riesgos y evalúen su viabilidad.

La próxima semana, cada equipo presentará su trabajo, explicando cómo llegó a sus conclusiones y qué decisiones tomó.

Como se ha dicho desde el inicio del semestre, en metodologías tradicionales se planifica todo el proyecto desde el inicio. En metodologías ágiles como Scrum, la planificación se hace por partes, a través de sprints.

Esto no significa que la planificación desaparezca, sino que se vuelve más flexible y continua.

Para adaptar todo lo mencionado anteriormente a Scrum, haremos lo siguiente:

Objetivo del proyecto.

En el enfoque tradicional se define un objetivo general del proyecto. En Scrum esto se divide en dos niveles:

- la visión del producto (qué problema resuelve el sistema)
- el objetivo del sprint (qué se logrará en ese periodo corto)

Por ejemplo: sistema escolar

Sprint 1: implementar inicio de sesión y registro de usuarios

Estimación de tiempo

En el enfoque tradicional se hace un cronograma completo del proyecto. En Scrum el tiempo se organiza en sprints, que son periodos cortos de trabajo (por ejemplo, 1 a 4 semanas).

Deben contemplar

- cuánto dura su sprint (por ejemplo, 2 semanas)
- qué se trabajó durante ese tiempo

No se necesita planear todo el proyecto, solo el sprint.

Estimación de costos

En Scrum no se trabaja tanto con costos detallados, pero sí con el esfuerzo del equipo. Se puede adaptar diciendo que el costo depende del tiempo invertido por el equipo.

Por ejemplo: 4 integrantes trabajando durante 2 semanas representan el esfuerzo del sprint.

Estimación de personal

En Scrum esto se traduce en roles:

- Product Owner (quien define lo que se necesita)
- Scrum Master (quien coordina el proceso)
- Equipo de desarrollo (quienes construyen el sistema)

Para su presentación ustedes indicaron quién cumple cada rol dentro del equipo.

Análisis de riesgos

En Scrum no se hace un documento formal de riesgos como en el enfoque tradicional, pero sí se identifican problemas durante el sprint. Estos se conocen como **impedimentos**.

Deben prestar atención:

- problemas que tuvieron durante el sprint
- dificultades técnicas o de organización

Ejemplos:

- falta de conocimiento en una tecnología
- errores en el sistema
- mala estimación del tiempo

Viabilidad del proyecto

En Scrum la viabilidad se demuestra con resultados reales. Es decir, con funcionalidades que ya están funcionando.

- qué parte del sistema ya funciona al finalizar el sprint

A este punto ustedes ya cuentan con el Product Backlog (historias de usuario) y ya presentaron su Sprint 1.

El objetivo ahora se convierte en el objetivo del sprint. El tiempo se organiza en sprints. El equipo se organiza en roles. Los riesgos se identifican como problemas durante el sprint.